AI-Queryable Health Data System

The scenario

Build a small end-to-end system that generates synthetic personal health metric data, lands it somewhere queryable, and (optionally) exposes a query endpoint with a natural-language layer on top.

You build the whole stack. Pick one or more health metrics you find interesting — heart rate, steps, sleep stages, workouts, blood oxygen, weight, whatever. Think about what realistic data from this domain actually looks like: per-user streams, irregular sampling, gaps when the device is off, occasional bursts during workouts, day/night patterns. Schema design is part of the exercise.

Core requirements

1. **Generate** a synthetic event stream of health metrics for a population of synthetic users. Volume, sampling rate, and realism are your call — pick something defensible and justify it. Containerized.
2. **Ingest** the events. Streaming or batch — your call, justify it.
3. **Store** the data in whatever shape and format makes sense for the access patterns you expect. Storage layer, table format (if any), file format, partitioning — all your call, justify it.
4. **Query** the data with an engine of your choice, justify it.
5. `docker compose up` boots the full system on a clean machine.
6. **README** with an architecture diagram, key decisions, and tradeoffs.

Bonus

Not required — pick them up if you want to show extra range. Tiered, so attempt them in order.

- **Bonus 1 — Query endpoint**: an HTTP route that takes a query and returns results. Query shape (SQL, structured filters, params — your call), request/response design, and error handling are part of the exercise.
- **Bonus 2 — Natural-language layer** (builds on Bonus 1): extend the endpoint to also accept NL questions (e.g. "*what was my average resting heart rate last week?*", "*which users had the most disrupted sleep in October?*") and return SQL + results or a direct answer. Any hosted LLM or local model is fine — your call, justify it. Skip if it would require paid API access you'd rather not cover.

Going deeper

Beyond the core requirements, pick one or two areas where you go deeper than strictly necessary — wherever your interests and strengths take you. We read those parts most carefully. Surface the depth in the README so we know where to look.

On AI usage

We expect you to use AI tooling heavily — Claude Code, Cursor, Copilot, whatever you use day-to-day. We're hiring engineers who leverage AI well, not engineers who avoid it. In your README, briefly cover:

- Which parts did you delegate to AI?
- Where did you override or correct it?
- What would you not trust AI to do here, and why?

This is a senior-judgment signal, not a gotcha. Be honest.

Submission

- **Public GitHub repository.**
- `docker compose up` runs cleanly on a fresh machine with documented prerequisites.
- README covering architecture, decisions, and the AI-usage notes above.
- One closing paragraph: **"If you had two more weeks, what would you build next?"** — this matters as much to us as the code.

Time expectation

You have **5 working days** from receiving the challenge to deliver. Within that window, spend as much (or as little) time on it as you like — we don't reward grinding, and we'd rather see a smaller, cleaner system than a sprawling one.

How we evaluate

- Sound architectural choices given the constraints.
- Working, runnable code.
- Clear thinking about tradeoffs.
- Senior judgment on where to spend depth vs. where to keep it minimal.
- Evidence of effective AI leverage.